



비전공자도 됩니다 - 4주 커리큘럼과 그 뒤에 이력서를 넣는 순서

코딩을 한 줄도 안 써본 상태에서 시작해, 인터넷에 배포된 프로젝트 1개와 그것을 스스로
설명할 수 있는 상태까지

이 페이지의 차례

1. 이 책이 다루는 것
2. 쓴 사람
3. 이 책이 주는 것과 주지 않는 것
4. 시작 전 준비물
5. 전체 목차

이 책을 읽는 법

1. 이 책이 다루는 것

이 책은 **AI 한줄일기**라는 앱 하나를 처음부터 만들어 인터넷에 올리고, 그 결과물을
이력서와 포트폴리오에 실어 지원을 시작하는 데까지를 다룹니다. 하루 한 줄 일기를 쓰면
AI가 코멘트를 달아주는, 기능이 몇 개 안 되는 앱입니다. 화면(React), 서버(Node.js),
데이터베이스(Supabase)가 한 줄로 이어지고 외부 AI API가 붙는 구조라서, 기능 개수는
적어도 웹 서비스가 돌아가는 뼈대는 전부 들어 있습니다.

그 앱을 만드는 도구는 **CLI 기반 AI 코딩 툴**입니다. 코드를 직접 타이핑하는 대신
지시문을 적어 AI에게 짜게 하고, 나온 결과를 확인하고, 다시 지시하는 방식으로
진행합니다. 그래서 이 책은 문법을 외우는 데 지면을 거의 쓰지 않습니다. 대신 무엇을
만들지 정리하는 법, **데이터가 어디서 어디로 흐르는지 읽는 법**, 막혔을 때 어디가 막힌
건지 구분하는 법, 그리고 완성된 것을 남에게 설명하는 법에 지면을 씁니다. 면접에서
질문받는 것도 그쪽입니다.

2. 쓴 사람

전공자였지만 반쪽짜리였습니다. 정보통신공학과라 회로가 메인이었고, 프로그래밍은 전공 안에서도 걸가지였습니다. 거기에 학점은 2.2였습니다. 공부를 하나도 안 했으니깐요. 2023년에 사실상 백지에서 시작해, 3년 만에 근로소득과 외주 수익을 합쳐 세전 1억을 넘겼습니다.

이 이력에서 중요한 건 숫자가 아니라 앞쪽입니다. 전공 수업 대부분에서 D를 받았으니, 시작 시점의 지식은 비전공자와 다를 게 없었습니다. 비전공자도 되는 게 아니라, **전공 여부가 애초에 결과를 가른 적이 없었다**는 뜻입니다. 예전엔 전공과 상관없이 이것도 쉽지 않았습니다. 클로드 같은 코딩 AI가 나온 지 어느덧 1년, 이제는 누구나 만들 수 있는 세상이 왔습니다. 갈린 건 무엇을 만들었고 그것을 설명할 수 있느냐였습니다. 이 책이 4주를 그 두 가지에만 쓰는 이유입니다.

3. 이 책이 주는 것과 주지 않는 것

먼저 이 책이 무엇을 약속하고 무엇을 약속하지 않는지 적어두겠습니다. 이 구분이 흐려지는 순간 이 책도 Part 1에서 뜯어보는 광고와 같은 물건이 됩니다.

이 책이 주는 것	이 책이 주지 않는 것
- 인터넷에 배포된 프로젝트 1개 (남의 컴퓨터에서도 열리는 링크와 깃허브 저장소) - 그 프로젝트를 처음부터 끝까지 내 입으로 설명할 수 있는 상태 - 지원을 시작할 자격, 그리고 어디에 어떻게 넣을지에 대한 전략(Part 3) - 그대로 활용할 수 있는 서브에이전트 패키지 (직접 만드는 3종에 서버·배포 2종을 더한 완성본 5종)	- 취업 보장. 이력서를 넣고 떨어지고 고치는 일은 4주가 끝난 다음 날부터 시작됩니다 - 연봉 보장. 이 책은 여러분이 얼마를 벌게 된다는 숫자를 제시하지 않습니다. 위 저자 소개의 소득은 저자 본인의 이력이고, 구성(근로 + 외주)과 세전 기준을 밝혔습니다. 그것과 여러분의 결과를 약속하는 것은 다른 얘기입니다 4주라는 기간의 보장. 4주는 약속이 아니라 기준점입니다

왼쪽 네 줄은 실제로 만들어지는 결과물을 기준으로 썼습니다. 결과물이 없으면 의미가 없기 때문입니다. 오른쪽 세 줄은 이 책이 확인해줄 방법이 없어서 약속하지 않는 것들입니다.

기간도 같은 기준으로 밝혀둡니다. 각 장 상단의 "예상 학습 기간"을 열세 개 다 더하면 **24일에서 37일**이 나옵니다. 4주(28일)는 그 범위의 가운데일 뿐이고, 상한도 하한도 아닙니다. 회사를 다니면서 저녁과 주말에만 붙는다면 더 걸립니다. 그건 뒤쳐진 게 아니라 정상입니다. 이 이야기는 Part 1의 6절 "그럼 이 책은 어떤가"에서 판별 체크리스트와 함께 더 자세히 다룹니다.

4. 시작 전 준비물

4주 동안 계정이 필요한 서비스는 일곱 개입니다. 어느 장에서 처음 필요해지는지 함께 적어둡니다.

계정이 필요한 것	처음 필요해지는 장	무료 여부
CLI 기반 AI 코딩 툴 Claude Code, Codex CLI 등(이 책의 실습은 Claude Code 기준)	1주차-2	계정 가입과 유료 구독이 있어야 실제로 작업이 됩니다. 4주 내내 씁니다
깃허브 코드를 올려두는 곳	1주차-2	무료로 가입합니다
Gemini API 키 AI 코멘트 기능	3주차-2	무료 할당량 범위 안에서 시작합니다
Supabase 데이터베이스	3주차-4	무료 플랜으로 시작합니다
Vercel 프론트엔드 배포	4주차-1	개인 프로젝트 규모에서는 무료 범위 안에서 시작합니다
Railway 백엔드 배포	4주차-1	체험 범위가 있고, 서버를 계속 켜두려면 결제수단 등록을 요구할 수 있습니다

계정이 필요한 것	처음 필요해지는 장	무료 여부
노션 프로젝트 상세 설명 페이지	4주차-2	무료로 가입합니다. 링크로 공유되는 문서 도구면 다른 것으로 대신합니다

- 4주 동안 필요한 계정과 그것이 처음 등장하는 장

첫날에 일곱 개를 다 만들 필요는 없습니다. 각 계정은 표에 적힌 장에서 처음 필요해지고, 그때 해당 장의 안내를 보면서 만들면 됩니다. 지금 당장 필요한 것은 1주차-2의 두 개, AI 코딩 툴과 깃허브뿐입니다.

AI 코딩 툴은 여러 종류가 있지만, 이 책의 모든 실습(특히 2주차의 서브에이전트·를 설정)은 **Claude Code** 기준으로 쓰여 있습니다. 그대로 따라가려면 Claude Code를 고르는 편이 낫습니다. 다른 도구를 선택할 수도 있는데, 그 경우 무엇이 달라지는지는 1주차-2에서 다룹니다.

그리고 이 과정에는 **돈이 듭니다**. 위 표의 "무료"는 정해진 사용량까지라는 뜻이고, 그 선을 넘으면 서비스가 멈추거나 요금이 청구됩니다. 가장 큰 지출은 CLI 기반 AI 코딩 툴 구독이며, 나머지를 다 합쳐도 그 하나에 미치지 못합니다. 비용이 생길 수 있는 다섯 항목이 무엇이고 각각 언제부터 필요한지는 4주차-1의 6절 "4주 동안 실제로 드는 돈"에 표로 정리되어 있습니다. 요금제는 자주 바뀌므로 금액은 여기서도 그 표에서도 못 박지 않습니다.

컴퓨터는 윈도우든 맥이든 상관없습니다. 다만 설치 방법이 운영체제마다 갈리는 지점이 있어서, AI에게 물을 때 "**나는 윈도우를 쓴다**"처럼 자기 운영체제를 질문 맨 앞에 적어야 합니다. 이 한 줄이 있고 없고에 따라 돌아오는 답이 완전히 달라집니다. 같은 내용을 1주차-2에서 실제 지시문과 함께 다룹니다.

5. 전체 목차

본문은 열다섯 장입니다. 앞뒤로 판별과 취업 전략이 한 장씩 붙고, 가운데 열세 장이 실제 제작 과정입니다.

시작하기 전

장	다루는 것
Part 1. 그거, 진짜예요?	과장광고인지 아닌지 스스로 판별하는 법. 판별 체크리스트를 만들고, 그 체크리스트로 이 책 자체를 먼저 검증합니다

1주차 - 개념과 도구를 손에 친다

서버와 API가 무엇인지 그림으로 잡고, AI 코딩 툴을 실제로 돌려 첫 결과물을 만듭니다.

장	다루는 것	이 장에서 얻는 것
1주차-1. 개념 잡기	서버, 클라이언트, API가 뭔지 그림으로 이해하고 개발환경을 세팅한다	-
1주차-2. CLI 툴 사용법	터미널에서 AI 코딩 도구를 실행해서 내 프로젝트 폴더의 코드를 직접 짜게 만든다	-
1주차-3. 미니 실습	지시문 → AI 코드 생성 → 확인 → 재지시, 이 반복 루프로 내 포트폴리오의 첫 버전을 만든다	portfolio.html - 내 포트폴리오 1차 버전

2주차 - AI를 다루는 법을 익힌다

코드를 더 짜는 주가 아니라, AI에게 무엇을 어떻게 시킬지를 정리하는 주입니다.

장	다루는 것	이 장에서 얻는 것
2주차-1. AI와 함께 기획하기	코드 한 줄 쓰기 전에, 무엇을 만들지부터 정리한다	기획.md - AI가 읽는 프로젝트 설계도
2주차-2. 서브에이전트 만들어 역할 나누기	명령을 내리는 AI 하나에 실제로 일하는 AI 여럿을 붙이는 구조를 익힌다	서브에이전트 파일 2개 - 코드 짜는 에이전트와 검토 에이전트
2주차-3. AI 도구에 규칙과 능력 붙이기	룰과 스킬로 AI에게 규칙을 주고, MCP로 AI가 닿는 범위를 넓힌다	CLAUDE.md - 내 프로젝트 전용 AI 규칙 파일

3주차 - 직접 만든다

서버, API, 화면, 데이터베이스를 하나씩 세우고 마지막에 한 줄로 잇습니다. 이 주가 끝나면 앱이 내 컴퓨터에서 돌아갑니다.

장	다루는 것	이 장에서 얻는 것
3주차-1. Node.js 서버 띄우기	요청이 오면 응답하는 프로그램을 눈으로 확인한다	Node.js + Express - 실제로 돌아가는 서버
3주차-2. API 개념 잡기	API가 정확히 뭔지 확실히 잡고, 그 예로 Gemini를 내 서버에 붙여본다	Gemini API - 외부 API 연동 경험
3주차-3. React로 화면 뼈대 세우기	레고 블록처럼 화면을 컴포넌트로 쪼개는 법	React + 리액트 에이전트 - 실제로 돌아가는 화면과 화면 담당 서브에이전트
3주차-4. 화면과 서버, 데이터베이스 잇기	내가 입력한 값이 서버를 거쳐 DB에 쌓이고, 다시 화면에 나타나기까지	DB 연결 - 실제로 저장하고 꺼내오는 전체 흐름

4주차 - 배포하고 취업 준비 상태로 만든다

내 컴퓨터에서만 돌던 것을 남도 열 수 있게 옮기고, 그 결과물을 이력서에 실을 수 있는 형태로 다듬은 뒤, 스스로 설명할 수 있는지 점검합니다.

장	다루는 것	이 장에서 얻는 것
4주차-1. 배포의 원리	내 컴퓨터 안에서만 돌던 프로젝트를, 아무나 접속할 수 있는 곳으로 옮긴다	배포 주소 - 남의 컴퓨터에서도 열리는 링크
4주차-2. 포트폴리오 정리	완성한 프로젝트를 이력서와 포트폴리오에 담을 수 있는 형태로 다듬는다	포트폴리오 PDF와 공유 링크 - 이력서에 첨부할 수 있는 형태
4주차-3. 코드 설명 점검	"이 코드, 본인이 짰 거 맞아요?"라는 질문 앞에서 무너지지 않는다	4주 완주 - 표지에서 약속한 목록을 손에 쥔 상태

완주 이후

장	다루는 것
Part 3. 완주 이후: 서류를 넣기 시작한다	이력서를 누가 읽는지 알고, 자소서엔 쓸 문장을 갖추고, 넣을 공고 100개를 고른다
별첨 자료	이 책과 함께 제공되는 파일과 사용법. 포트폴리오 템플릿을 포함합니다

4주 뒤 손에 남는 것

AI 한줄일기 - 남의 컴퓨터에서도 열리는 배포 링크 1개

- 포트폴리오 문서(portfolio.html)와 그 안에 들어간 프로젝트 링크 3종
- 깃허브 저장소 하나. 기획.md, 서버에이전트 파일 3개, CLAUDE.md가 함께 올라가
있습니다
- Node.js와 Express로 띄운 서버, Gemini API 연동, React 화면, DB 연결을 각각 직접
해본 경험
- 그 전부를 처음부터 끝까지 말로 설명할 수 있는 상태

이 책을 읽는 법

1주차부터 4주차까지 열세 개 장에는 그 장의 차례, 체크포인트, 요약, 그리고 문제 세
개가 들어 있습니다. 체크포인트를 스스로 설명하지 못하면 다음 장으로 넘어가지 말고 그
장을 다시 보십시오. 순서대로 읽어야 합니다. 앞 장의 결과물이 다음 장의 재료가 되기
때문에, 건너뛰면 재료가 없는 상태로 실습을 시작하게 됩니다. 그리고 막히는 순간에는
화면에 뜬 문구를 그대로 AI에게 붙여넣고 물어보십시오. 이 책이 처음부터 끝까지 쓰는
방식이 그것이고, 4주 뒤에도 계속 쓰게 될 방식입니다.

본문과 별개로 별첨 여섯 가지가 함께 제공됩니다. **별첨 1 정적 포트폴리오 템플릿**(내용만 갈아끼우는 완성형 문서), **별첨 2 에러 대처표**(막혔을 때 화면에 뜬 문구로 찾는 표), **별첨 3 프롬프트팩**(본문에 나온 지시문 모음), **별첨 4 주차별 체크리스트**(장마다 배우는 것과 확인할 것을 표로 정리해 4주 내내 옆에 두는 자료), **별첨 5 완성 레포**(4주 결과물의 정답 코드와 서브에이전트 참고 자료), **별첨 6 저자의 실제 공통 자소서**(구조를 보는 자료이지 베끼는 자료가 아닙니다)입니다.

Part 1. 그거, 진짜예요?

과장광고인지 아닌지 스스로 판별하는 법

이 장의 차례

1. 이런 문구, 본 적 있으시죠?
 2. 화면 속 마법: 직원이 돌아다니는 그 데모
 3. 숫자로 보는 광고 vs 현실
 4. 판별 체크리스트
 5. 업계는 이미 이 방향으로 움직이고 있다
 6. 그럼 이 책은 어떤가
-

결론

"코딩 몰라도 개발자 취업 가능", "AI가 대신 다 해준다", "몇 주 만에 서비스 하나 완성". 요즘 이런 문구를 안 보고 지나가기가 더 어렵습니다. 광고인지 진짜인지 판단이 안 서서 이 책을 집어 든 분들도 많을 겁니다.

결론부터 말하면, 이 문구들 중 상당수는 과장이거나 거짓입니다. 하지만 그렇다고 "그러니까 나는 못 하겠다"로 결론 내리면 안 됩니다. 저 광고들이 틀린 건 **방법**이지, "가능하다"는 사실 자체가 아니기 때문입니다. 이 장에서는 어떤 문구를 의심해야 하는지, 왜 의심해야 하는지 하나씩 뜯어보겠습니다.

1. 이런 문구, 본 적 있으시죠?

특정 업체를 저격하려는 게 아닙니다. 이 시장에서 반복적으로 등장하는 표현 패턴을 모아봤습니다.

"코딩을 몰라도 AI가 대신 개발해줍니다. 아이디어만 말하면 끝."

- 온라인 강의 상세페이지에서 흔히 보이는 문구

"3시간 만에 만든 앱으로 월 1억 넘게 벌었습니다."

- 수익 자랑형 광고에서 흔히 보이는 문구, 검증 불가능한 숫자

"90일 완성, 대학원 갈 필요 없이 억대 연봉."

- 기간 단축 + 고연봉을 함께 붙이는 전형적 조합

공통점이 보이시나요? 기간은 비정상적으로 짧고, 결과는 비정상적으로 큼니다. 그리고 그 사이에 있어야 할 "어떻게"는 항상 빠져 있습니다.

내용을 들여다봐도 마찬가지입니다. 시중 AI 강의 중 상당수는 강의 자료 자체가 AI에게 직접 물어보면 나오는 정보를 나열해둔 것에 그칩니다. 그 정보를 실습으로 직접 써서 완성까지 가보는 과정 없이, 개념 설명만 반복하는 겁니다. 그럴 바에는 유튜브의 코딩 기초 채널에서 코드가 실제로 어떻게 흘러가는지 보는 편이 낫습니다. 그런 강의는 굳이 볼 필요가 없습니다.

2. 화면 속 마법: 직원이 돌아다니는 그 데모

가장 흔한 눈속임 중 하나를 짚고 넘어가겠습니다. 화면에 캐릭터들이 사무실을 돌아다니면서 "일하는 것처럼" 보이는 데모입니다. 비개발자가 보면 이런 반응이 나옵니다.

"와, 진짜 직원을 부리네?"

실제로 화면 뒤에서 벌어지는 일은 이렇습니다.



캐릭터가 걸어다니는 애니메이션은 이 흐름에서 **아무 역할이 없습니다**. API 호출 한 번 하고 결과를 받는 걸 화려하게 포장한 것뿐입니다. 움직임 자체는 기능이 아니라 연출이고, 연출에 들어간 시간과 비용은 실제 작동 원리를 이해하는 데 전혀 도움이 되지 않습니다.

기억할 것: **화면이 화려할수록 오히려 의심하십시오**. 진짜 실력 있는 결과물은 대부분 화면이 수수합니다. 화려한 쇼는 비전공자를 현혹시키는 게 목적인 경우가 많습니다.

3. 숫자로 보는 광고 vs 현실

공개된 후기와 커뮤니티 사례를 기간 기준으로만 비교하면 이렇습니다.

광고가 말하는 것	실제 성공 사례의 소요 기간
• 90일 완성, 연봉 6천 • 5주 만에 서비스 출시 • 3시간 만에 월 1억	• 비전공자 독학, 하루 15시간씩 6개월 • 국비과정 수료 후 3개월 취업준비 + 6번 이상 면접, 총 1년 • 부트캠프 8기 수료 후 조교 근무 거쳐 취업

그리고 이 실제 성공 사례들에는 공통점이 하나 더 있습니다. **"AI가 대신 해줬다"는 언급이 거의 없습니다**. 오히려 원리를 직접 이해하려고 오래 붙잡고 있었다는 이야기뿐입니다.

4. 판별 체크리스트

다음 중 두 개 이상 해당하면 그 광고는 의심하십시오.

- **기간이 비정상적으로 짧다** - 90일, 5주, 3시간처럼 결과물 난이도 대비 압도적으로 짧은 기간을 내세운다
 - **결과가 검증 불가능한 숫자다** - "월 1억", "억대 연봉"처럼 확인할 방법이 없는 숫자를 내세운다. 여기서 핵심은 숫자가 크다는 게 아니라 **검증이 안 된다는 것**이다. 누구의 소득인지, 무엇으로 이루어졌는지, 세전인지 세후인지, 증빙을 보여줄 수 있는지가 빠져 있으면 그 숫자는 미끼다
 - **"몰라도 된다"를 강조한다** - 이해가 필요 없다는 걸 장점으로 포장한다
 - **화면 연출이 기능보다 화려하다** - 캐릭터, 이펙트, 실시간 그래프 등 시각적 요소가 과하게 강조된다
 - **"어떻게"가 빠져 있다** - 과정 설명 없이 결과만 반복해서 보여준다
-

5. 업계는 이미 이 방향으로 움직이고 있다

이건 광고가 아니라 실제로 벌어진 일입니다. 2026년 7월, CJ올리브영은 경력 개발자 채용의 한 트랙(AI-First Track, 경력 2년 이상, 3개 직무 한정)에서 15년 넘게 개발자 채용에 이어져 온 코딩테스트를 없앴습니다. 대신 도입한 "AI 과제" 전형은 지원자가 AI 도구를 활용해 과제를 해결하는 과정을 다섯 가지 기준으로 평가합니다: 문제를 어떻게 정의했는지, AI를 어떻게 활용했는지(프롬프트 작성 과정 포함), 얼마나 반복해서 개선했는지, 결과를 어떻게 검증했는지, 최종 결과물이 무엇인지.

주목할 부분은 이 다섯 가지 기준 어디에도 **"얼마나 빨리 짰는가"**는 없다는 점입니다. 평가하는 건 속도가 아니라, AI와 협업해서 문제를 풀어내는 과정 전체를 설명할 수 있느냐입니다. 이 책이 처음부터 강조해온 것과 정확히 같은 방향입니다.

다만 정직하게 밝힙니다: 이건 한 회사가 2026년 7월 처음 시도한 특정 경력직 트랙의 사례입니다. 모든 회사가 이렇게 바뀌었다는 뜻은 아니고, 신입 채용까지 이 방식이 퍼졌다는 근거도 아직 없습니다. 다만 업계가 어느 방향을 보고 있는지는 이 사례 하나로도 분명히 보입니다.

6. 그럼 이 책은 어떤가

방금 만든 체크리스트의 첫 항목은 "기간이 비정상적으로 짧다"였습니다. 그런데 이 책은 **4주**를 이야기합니다. 앞에서 뜯어본 "90일 완성"보다도 짧습니다. 그러니 이 책도 같은 기준으로 검증받고 넘어가겠습니다.

갈리는 지점은 기간 숫자가 아니라 **그 기간이 끝났을 때 손에 뭐가 남는지**입니다.

"90일 완성"이 파는 것	이 책의 4주가 파는 것
• 90일 뒤 = 취업 완료 • 연봉 숫자를 함께 붙임 • 끝났는지 확인할 방법이 없음	• 인터넷에 배포된 프로젝트 1개 • 그 프로젝트를 내 입으로 설명할 수 있는 상태 • 링크를 눌러보고 한번 말해보면 바로 확인됨

이 책의 4주는 **취업 완료**가 아닙니다. 지원을 시작할 자격을 갖추는 데까지입니다. 이력서를 넣고, 떨어지고, 그 결과를 보고 다시 고치는 일은 4주가 끝난 다음 날부터 시작됩니다(그 전략은 Part 3에서 다룹니다). 그 부분까지 4주 안에 넣어서 팔면 이 책도 앞에서 뜯어본 광고와 같은 물건이 됩니다.

판별 체크리스트 다섯 항목에 이 책이 어떻게 답하는지 그대로 적어두겠습니다.

판별 항목	이 책은 여기에 어떻게 답하는가
기간이 비정상적으로 짧다	4주가 만드는 결과물을 "배포된 프로젝트 1개 + 그것을 설명할 수 있는 상태"로 좁혀서 명시한다. 취업 완료는 4주 안에 넣지 않았다
결과가 검증 불가능한 숫자다	독자가 얼마를 벌게 된다는 숫자를 제시하지 않는다. 저자 소개에 소득이 한 줄 나오지만 저자 본인의 이력이고, 구성(근로 + 외주)과 세전 기준을 밝혔다. 증빙 제시 여부까지는 적어두지 않았지만, 그 정도만으로도 누구의 소득인지· 무엇으로 이루어졌는지조차 밝히지 않는 완전히 무근거인 숫자와는 다르다. 그리고 이 책이 약속하는 결과물은 배포 링크와 깃허브 주소라서 누구나 눌러서 확인할 수 있다
"몰라도 된다"를 강조한다	반대로 간다. 마지막 장 전체를 "만든 것을 스스로 설명해보는 점검"에 쓴다. 설명하지 못하면 완주로 치지 않는다
화면 연출이 기능보다 화려하다	4주 동안 만드는 앱은 한 줄 일기를 저장하고 AI 코멘트를 한 줄 받는 것이 전부다. 기능 개수를 늘리지 않는다
"어떻게"가 빠져 있다	본문 대부분이 "어떻게"다. 실제로 입력할 지시문과 터미널 명령을 그대로 신는다

마지막으로 4주라는 숫자 자체입니다. 4주는 절대 기준이 아닙니다. 이 책의 각 장
상단에는 "예상 학습 기간"이 적혀 있는데, 열세 개 장을 다 더하면 **24일에서 37일**
사이가 나옵니다. 4주(28일)는 그 범위의 가운데일 뿐이고, 상한도 하한도 아닙니다.
회사를 다니면서 저녁과 주말에만 붙는다면 더 걸립니다. 그건 뒤흔친 게 아니라
정상입니다.

이 책이 하는 약속은 하나입니다. **4주 뒤에 남의 화면에서도 열리는 링크 하나와,
그것을 처음부터 끝까지 설명할 수 있는 상태를 만든다.** 지켜졌는지는 4주차가 끝나는
날 여러분이 직접 판정하면 됩니다. 이 책도 그렇게 검증받겠습니다.

결론

저 광고들이 거짓말을 하는 건 맞습니다. 다만 거짓말의 핵심은 기간 숫자 자체가 아니라, 그 기간이 끝났을 때 손에 뭐가 남는지를 바꿔치기한다는 데 있습니다. "90일 뒤에 취업이 끝나 있다"는 확인할 방법이 없는 약속이고, "4주 뒤에 배포된 프로젝트 하나와 그걸 설명할 수 있는 상태가 남는다"는 눌러보면 바로 확인되는 약속입니다.

이 책은 뒤쪽만 약속합니다. 그 상태까지 가면 무전공에 완전히 처음 시작한 사람도 개발자 채용에 지원할 자격을 갖추니다. 다음 장부터는 그 방법을 그대로 보여드리겠습니다.

1주차-1. 개념 잡기

서버, 클라이언트, API가 뭔지 그림으로 이해하고 개발환경을 세팅한다

오늘 배울 것 클라이언트-서버-API의 요청/응답 구조를 이해하고, 터미널·코드 에디터·언어라는 개발환경 개념을 잡습니다

예상 학습 기간 2~3일

이 장의 차례

1. 홈페이지를 뜯어보면 세 조각이다
 2. 세 조각을 AI에게 확인받고, 실제 사이트에 적용해보기
 3. API는 "DB에서 뭘 가져올지 미리 정해놓은 것"이다
 4. 왕복 흐름 그림으로 보기
 5. 개발환경이 왜 필요한가
-

헛갈리는 용어부터 정리하고 갑니다 실습 과제 체크포인트 요약 문제 풀러 가기

이번 구간이 끝나면: 서버와 클라이언트가 뭔지, 인터넷으로 뭔가를 "요청하고 응답받는다"는 게 실제로 무슨 뜻인지 그림을 그리며 설명할 수 있게 됩니다. 그리고 앞으로 매일 쓸 개발환경(터미널, 코드 에디터, 언어)이 각각 뭘 하는 도구인지 정리해둡니다.

정민(27살, 개발과 무관한 일을 하다가 진로 전환 중)이 하던 일의 상당 부분은 정해진 규칙을 사람이 손으로 반복하는 작업이었습니다. 문서를 정리하고, 데이터를 옮겨 적고, 같은 양식의 보고서를 조건만 바꿔 다시 만드는 일이 매번 되풀이됐습니다. 이

흐름을 도구로 만들면 되겠다고 생각했지만, 그때 정민은 요청하는 쪽이었지 만드는 쪽이 아니었습니다.

그래서 직접 만들어보기로 하고, 하루 한 줄 일기를 쓰면 AI가 코멘트를 달아주는 **AI 한줄일기**를 첫 프로젝트로 잡았습니다. 그런데 "서버", "클라이언트", "API" 같은 말부터 막혀 시작도 못 하고 있습니다.

"다들 아는 것처럼 말하는데 나는 이 단어들이 대체 뭘 가리키는 건지도 모르겠다"

- 정민의 생각

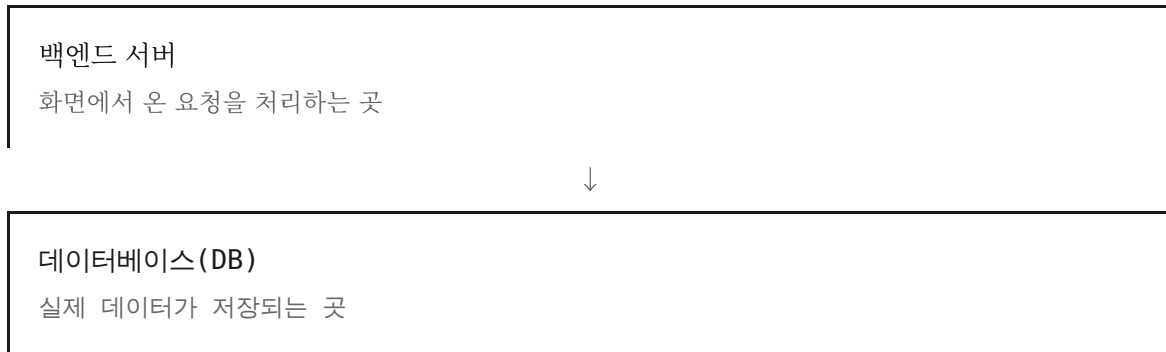
오늘은 이 세 단어를 그림 한 장으로 정리하고, 터미널에 첫 명령을 직접 쳐봅니다.

1. 홈페이지를 뜯어보면 세 조각이다

개발 공부를 시작하면 "서버", "클라이언트", "프론트엔드", "백엔드" 같은 말이 한꺼번에 쏟아집니다. 이걸 억지로 외우려 하지 말고, 정민이 만들려는 홈페이지(AI 한줄일기) 하나를 그림으로 뜯어보겠습니다.

AI 한줄일기 (브라우저 주소창)
사용자 눈에 보이는 부분 오늘의 한 줄 입력창, 저장 버튼, AI 코멘트 이 화면 전체 = 프론트엔드

↓ 화면 뒤에 숨어서 일하는 부분 (사용자 눈에 안 보임)



홈페이지 하나는 사실 이 세 조각이 합쳐진 겁니다. 사용자 눈에 보이는 건 맨 위 화면 (프론트엔드)뿐이고, 백엔드 서버와 DB는 화면 뒤에 숨어서 조용히 일합니다.

화면(프론트엔드)에서 버튼을 누르면, 그 클릭이 인터넷을 타고 백엔드 서버로 **요청(request)**을 보냅니다. 서버는 필요하면 DB를 열어보고, 처리한 결과를 다시 화면으로 **응답(response)**해서 돌려줍니다.

우리가 매일 쓰는 모든 웹사이트와 앱은 사실 이 요청과 응답의 왕복을 반복하고 있을 뿐입니다.

정민이 매일 쓰는 SNS도 다르지 않습니다. 피드에 뜨는 사진과 좋아요 버튼은 프론트엔드가 그려주는 화면입니다. 좋아요를 누르면 그 클릭이 요청이 되어 SNS의 백엔드 서버로 날아가고, 서버는 "이 사진의 좋아요 개수를 하나 늘려라"는 규칙대로 DB에 저장된 숫자를 고쳐줍니다. 화면에 찍힌 숫자가 바로 바뀌는 것처럼 보이는 것도, 사실은 이 왕복이 끝나고 그 결과를 화면이 다시 받아서 그린 것입니다.

여기서 흔히 헷갈리는 것 하나는 "서버"라는 말이 가리키는 게 정확히 뭐냐는 것입니다. 서버는 특정 기계 한 대를 가리키는 말이 아니라, 요청이 오면 처리해서 응답해주는 프로그램을 가리킵니다. 지금은 이 프로그램이 정민의 노트북 안에서만 돌아가고 있어서 정민 말고는 아무도 접속할 수 없지만, 나중에 이 프로그램을 인터넷에 연결된 다른 컴퓨터에 올려서 계속 켜두면 다른 사람도 그 서버에 요청을 보낼 수 있게 됩니다.

헛갈리는 용어부터 정리하고 갑니다

- 프론트 = 프론트엔드 = 클라이언트 = 화면 - 전부 "사용자가 눈으로 보는 부분"을 가리키는 같은 말입니다. 이 책에서도 문맥에 따라 섞어 쓰지만, 항상 같은 대상을 가리킵니다.
- 주의: "클라이언트"에는 또 다른 뜻도 있습니다. 일상에서 "클라이언트"는 흔히 "고객"이라는 뜻으로도 씁니다(예: "클라이언트 미팅"). 이 책에서 "클라이언트"라고 하면 항상 화면(프론트엔드) 얘기이지, 고객이라는 뜻이 아닙니다.
- 백엔드 = 서버 - 이것도 같은 대상을 가리키는 말입니다.
- 지시문 = 프롬프트 - 이 책은 AI에게 주는 글을 지시문이라고 부르지만, 개발 현장에서는 프롬프트라고 부릅니다. 같은 것을 가리키는 말입니다.

2. 세 조각을 AI에게 확인받고, 실제 사이트에 적용해보기

방금 그림으로 본 세 조각 구조는 눈으로 보고 고개를 끄덕이는 것과, 내 말로 다시 설명할 수 있는 것이 다릅니다. CLI 기반 AI 코딩 툴을 대화 상대로 써서, 지금 이해한 걸 직접 말해보고 맞는지 확인받아보겠습니다.

[맥락] 방금 프론트엔드-백엔드-DB 세 조각 구조를 배웠어

[요청] 내가 이해한 걸 말해볼 테니 맞는지 확인해줘

[조건] "화면에서 버튼을 누르면 그 클릭이 요청이 되어 서버로 가고, 서버는 필요하면 DB를 열어보고 처리한 결과를 응답으로 화면에 돌려준다" - 이 설명에 틀린 부분이 있으면 어디가 틀렸는지 짚어줘

- 세 조각 개념을 내 말로 설명해보고 AI에게 확인받는 지시문 예시

이렇게 스스로 설명해보고 AI에게 확인받는 과정을 거치면, 어느 부분이 아직 흐릿한지가 드러납니다. 다음은 이 구조를 한 번도 본 적 없는 실제 사이트 기능에 그대로 대입해보는 지시문입니다.

【맥락】 방금 프론트엔드-백엔드-DB 구조를 배웠어

【요청】 쇼핑몰에서 상품을 장바구니에 담는 기능이 이 세 조각 중 각각 어디서 처리되는지 설명해줘

【조건】 화면(프론트엔드)이 하는 일, 서버(백엔드)가 하는 일, DB가 하는 일을 나눠서 알려줘

- 실제 사이트 기능을 세 조각에 대입해보는 지시문 예시

이 두 지시문의 목적은 다릅니다. 첫 번째는 내가 설명한 것이 맞는지 확인받는 것이고, 두 번째는 내가 배운 틀을 낯선 상황에 직접 적용해보는 것입니다. 이 두 가지를 왔다갔다 하는 습관은 앞으로 새로운 개념을 배울 때마다 똑같이 쓸모가 있습니다.

3. API는 "DB에서 뭘 가져올지 미리 정해놓은 것"이다

정민의 AI 한줄일기 화면에서 사용자가 오늘 있었던 일을 한 줄 적고 [저장] 버튼을 클릭하는 순간을 예로 들어보겠습니다. 이때 프론트엔드-백엔드-DB, 각 층에서 실제로 무슨 일이 일어나는지 나눠서 보면 이렇습니다.

프론트엔드 (화면)
오늘 첫 서버를 띄웠다 저장 사용자가 이 입력창에 한 줄을 적고 [저장] 버튼을 클릭 → 백엔드의 /diary 주소로 "오늘 첫 서버를 띄웠다"는 글을 요청으로 보냄

↓ 이 요청이 백엔드로 도착함

백엔드

/diary 주소로 요청 받음

미리 정해둔 규칙: "diary 표에 오늘 날짜와 글을 새 줄로 저장해라"

INSERT INTO diary (date, text) VALUES ('7월 31일', '오늘 첫 서버를 띄웠다')

↓ 규칙대로 DB에 새 줄을 저장함

데이터베이스(DB)

규칙대로 처리한 결과, 표에는 실제로 이렇게 저장됩니다

date (날짜)	text (일기 내용)
7월 31일	오늘 첫 서버를 띄웠다

- 정민의 AI 한줄일기 API 형태 예시

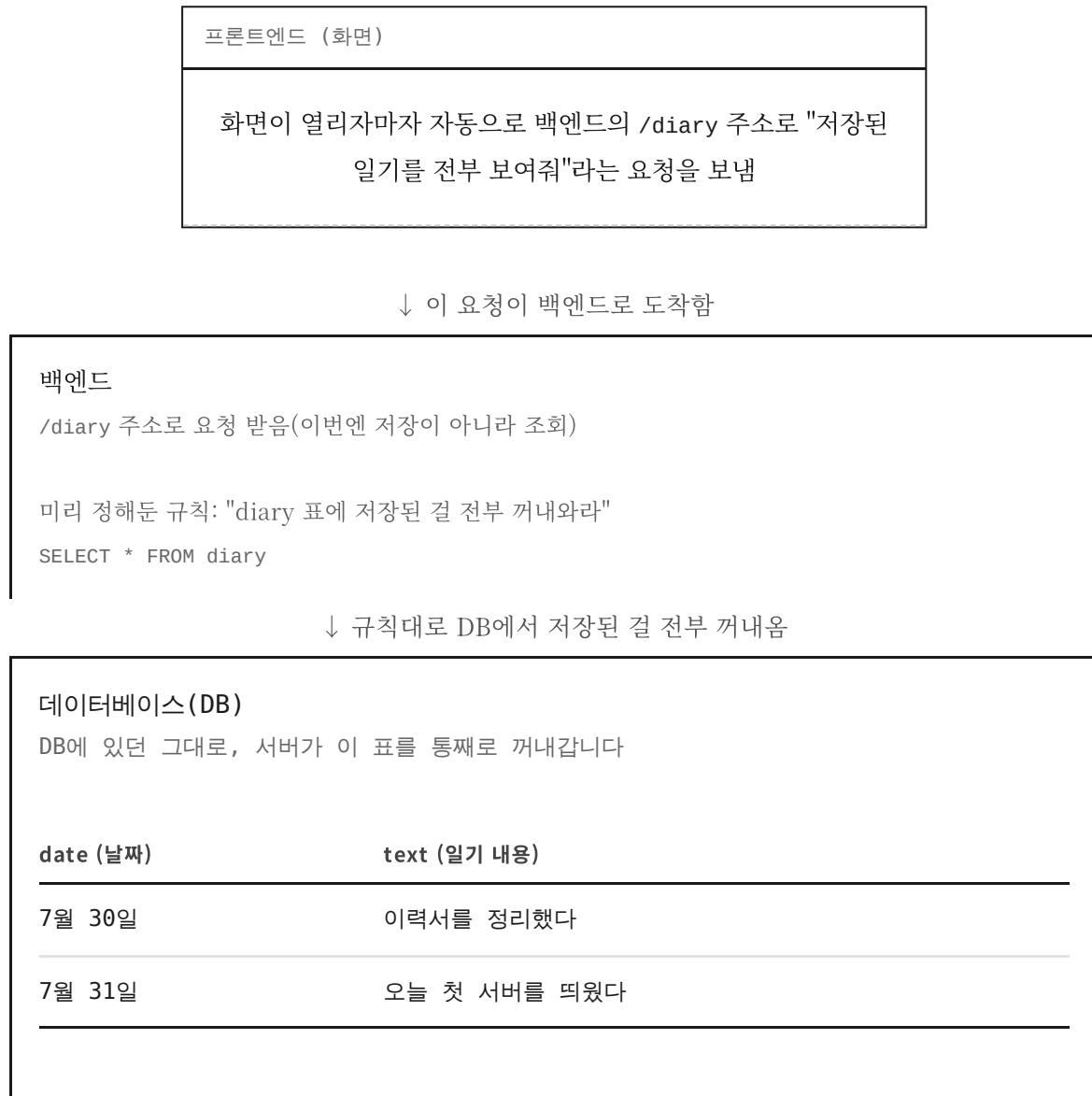
방금 본 INSERT INTO 같은 코드는 데이터베이스에 말을 거는 언어인 **SQL**입니다. 이 문법은 3주차에서 다시 다룹니다. 지금은 "데이터베이스에 이런 식으로 말을 건다"는 정도만 봐두면 됩니다.

이렇게 미리 정해놓은 규칙 자체가 **API**입니다. 쉽게 말하면, API는 "이 주소로 요청이 오면 DB의 이 항목을 어떻게 다룰지(저장할지, 가져올지) 미리 짜둔 규칙"입니다. 위 예시는 일기를 **저장**하는 API였고, 반대로 "지난 일기 목록 보여줘"라는 요청이 오면 DB에서 **가져와서** 돌려주는 API가 따로 정해져 있는 식입니다. 프론트엔드는 이 규칙까지 알 필요 없이, 정해진 주소로 요청만 보내면 백엔드가 알아서 그 규칙대로 DB를 다뤄줍니다.

참고로 앱의 핵심 기능인 AI 코멘트(comment)도 나중에는 이 표에 한 칸으로 함께 저장됩니다. 코멘트를 만들어 붙이는 방법은 3주차에서 배웁니다. 지금은 "일기가 이렇게 저장된다"까지만 알면 됩니다.

여기서 중요한 건, API를 "부르는 방법"(호출 방식)은 상황마다 다르다는 점입니다. 위 예시는 버튼 클릭이지만, 화면이 열리자마자 자동으로 부르는 경우도 있고, 검색창에 글자를 입력할 때마다 부르는 경우도 있습니다. "언제 부르는지"는 매번 다르지만, "이 주소로 오면 DB를 이렇게 다룬다"고 미리 정해둔 규칙 자체는 항상 똑같습니다.

방금 말한 "화면이 열리자마자 자동으로 부르는 경우"를 그림으로 보면 이렇습니다.



- 정민의 AI 한줄일기, "가져오는" API 형태 예시

같은 /diary 주소라도 어떤 요청이 오느냐에 따라 백엔드는 다른 규칙을 실행합니다. 저장해달라는 요청이 오면 앞서 본 INSERT 규칙을, 가져와달라는 요청이 오면 방금 본 SELECT 규칙을 실행하는 식입니다. 요청의 종류를 부르는 정확한 이름과, 화면과

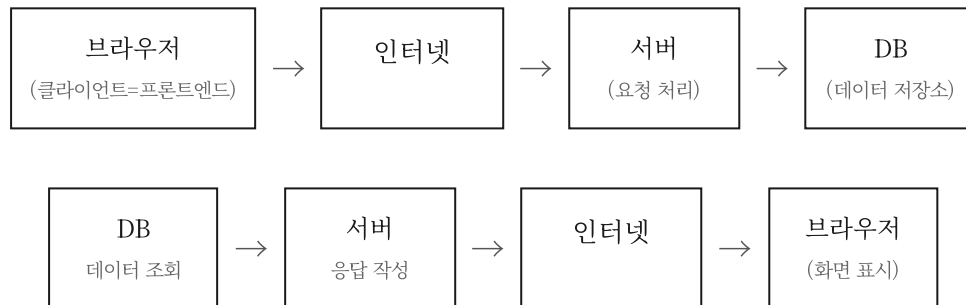
서버와 DB가 이 두 규칙을 실제로 어떻게 연결해서 쓰는지는 3주차-4(화면과 서버, 데이터베이스 잇기)에서 코드로 직접 만들어봅니다.

API를 어렵게 외울 필요는 없습니다. "이 주소로 요청이 오면 DB에서 뭘 가져올지 미리 정해둔 규칙" 정도로만 이해하고 넘어가도 앞으로 실습하는 데 전혀 문제가 없습니다.

- 이 책에서 반복적으로 쓰이는 최소 정의

4. 왕복 흐름 그림으로 보기

우리가 웹사이트에서 뭔가를 누를 때마다 화면 뒤에서 이런 일이 벌어집니다.



로그인처럼 작은 기능이라면, 이 왕복 전체가 **0.1초도 안 걸려서** 순식간에 끝납니다.

그래서 우리 눈에는 버튼을 누르자마자 바로 결과가 뜨는 것처럼 보이는 겁니다.

그림을 보면 "브라우저가 DB에 바로 접속하면 안 되나?"라는 의문이 들 수 있습니다.

DB에는 다른 사용자의 정보까지 전부 들어 있어서, 누구나 직접 접속할 수 있게 열어두면 원하는 데이터를 마음대로 가져가거나 지울 수 있게 됩니다. 그래서 브라우저는 DB에 직접 닿지 않고 항상 서버를 거쳐서만 요청을 보냅니다. 서버는 그 요청이 정당한 요청인지, 어디까지 보여줄지를 판단하는 관문 역할을 합니다.

DB(데이터베이스)는 서버가 참고하는 **창고**라고 생각하면 됩니다. 회원 정보, 게시물, 주문 내역 같은 데이터가 여기 쌓여 있습니다. 서버는 요청이 올 때마다 이 창고를 열어보고, 필요한 걸 꺼내서 응답에 담아 돌려줍니다.

정민이 4주 뒤에 완성할 AI 한줄일기도 다르지 않습니다. "오늘 일기 저장해줘"라는 요청이 오면 서버가 그 글을 DB에 저장합니다. 나중에 "지난 일기 목록을 보여줘"라는 요청이 오면 서버는 DB에서 그 기록들을 꺼내 화면에 돌려줍니다. 오늘 익힌 이 왕복 구조가 앞으로 4주 동안 계속 반복해서 쓰일 기본 뼈대입니다.

5. 개발환경이 왜 필요한가

그림을 이해했으니 이제 실제로 이 왕복 구조를 만들 도구가 필요합니다. 세 가지만 있으면 됩니다. 왜 필요한지 개념만 잡고 가겠습니다. 설치 방법은 1주차-2에서 AI 도구에게 직접 물어보면서 진행합니다.

터미널

터미널은 컴퓨터에게 글자로 명령을 내리는 창입니다. 마우스로 아이콘을 클릭하는 대신 "이 폴더로 이동해", "이 프로그램을 실행해" 같은 명령을 타이핑으로 내립니다. 개발자와 AI 코딩 도구는 대부분 이 터미널을 통해 대화합니다.

터미널 켜는 법

맥: Cmd+Space → "터미널" 입력

윈도우: 시작 메뉴 → "터미널" 검색

에디터

사실 코드는 **메모장**으로 수정해도 똑같이 돌아갑니다. 컴퓨터 입장에서는 그냥 글자로 된 파일일 뿐입니다. 다만 메모장은 사람이 보기에 너무 불편합니다. **코드 에디터**는 그 코드를 사람이 보기 편하게 해주는 도구입니다. 코드를 읽기 쉽게 색깔로 구분해주고, 파일과 폴더 구조를 한눈에 보여줍니다. 대표적으로 Cursor나 VS Code가 있고, 다루는 언어나 팀 컨벤션에 따라 다른 에디터를 쓰기도 합니다. 다만 이 책처럼 CLI 기반 AI 코딩 툴(Claude Code 등)로 작업하면, 대부분의 일이 터미널 안에서 끝나기 때문에 코드 에디터를 직접 열어서 들여다볼 일이 별로 없을 수도 있습니다.

언어

세 번째는 **언어**입니다. 컴퓨터에게 시킬 일을 적는 방법에도 여러 종류가 있습니다. 백엔드(서버)를 만들 때 쓰는 언어만 해도 Node.js(자바스크립트), 파이썬, 자바 등 여러 가지가 있고, 프론트엔드 (화면)도 React, Vue, 혹은 그냥 HTML 같은 여러 방식이 있습니다. 이 책에서는 자바스크립트 (Node.js + React)를 씁니다. 국내엔 자바를 쓰는 회사도 많지만, 해외에서는 Node.js와 React 조합이 가장 유행하는 조합입니다. 정보도 많고, MVP나 바이브코딩엔 더 빠르게 구축되는 편이라 이 책에서 선택했습니다. 어떤 언어를 쓰는지 중요하지 않다는 뜻은 아닙니다. 다만 AI를 잘 활용하면 어떤 언어든 다룰 수 있게 됩니다. 그래서 지금 이 언어 하나에 갇히지 말고, 언어보다 AI를 얼마나 잘 부리는지에 집중하는 게 더 중요합니다.

지금 배운 세 가지(터미널, 에디터, 언어) 위에 앞으로 몇 가지 도구가 더 올라갑니다. 언제 등장하는지 미리 표로 잡아두면, 나중에 새 이름이 나올 때마다 낯설지 않습니다.

앞으로 등장하는 도구	쓰이는 곳	등장하는 장
Gemini API 키	AI 코멘트 기능	3주차-2
Supabase	데이터베이스 연결	3주차-4
Vercel	화면(프론트엔드) 배포	4주차-1
Railway	서버(백엔드) 배포	4주차-1

- 4주 동안 등장하는 개발 도구 로드맵

지금은 이 이름을 하나하나 외울 필요 없이, 표에 있는 순서대로 그때그때 하나씩 익히면 됩니다.

이 언어로 코드를 다 짰다고 해서 저절로 돌아가는 건 아닙니다. **그 코드를 실제로 "서버로 열어서" 돌리는 것도 똑같이 중요합니다.** 이걸 3주차-1("서버 띄우기")에서 자세히 다룹니다.

기억할 것: 세 가지 도구는 각자 역할이 다릅니다. 터미널은 명령을 내리는 창, 에디터는 코드를 보고 쓰는 곳, 언어(이 책에서는 자바스크립트)는 코드를 적는 문법입니다. 언어로 코드를 적는 것과, 그 코드를 실제로 돌아가게 여는 것(서버 실행)은 별개입니다. 이 역할 구분만 잡아도 설치 과정에서 헤매지 않습니다.

실습 과제

아직 CLI 기반 AI 코딩 툴을 설치하기 전이라도, 터미널 명령 몇 개는 지금 바로 손으로 쳐볼 수 있습니다.

1. 터미널(또는 명령 프롬프트)을 열어보세요.
2. `mkdir ai-study`라고 입력해서 `ai-study`라는 이름의 새 폴더를 만들어보세요. 화면에 아무 메시지도 뜨지 않으면 정상입니다.
3. `ls`라고 입력해서 `ai-study` 폴더가 생겼는지 목록으로 확인해보세요.
4. `cd ai-study`라고 입력해서 방금 만든 그 폴더로 이동해보세요.

- 1주차-1 실습: `mkdir`로 폴더 만들고 `cd`로 이동해보기

체크포인트

다음 항목을 스스로 설명할 수 있으면 통과입니다.

- 클라이언트와 서버가 뭔지 내 말로 설명할 수 있는가 - 홈페이지 그림을 안 봐도, 누가 요청하고 누가 처리하는지 구분해서 말할 수 있는가
- API가 왜 필요한지 설명할 수 있는가 - "이 주소로 요청이 오면 DB에서 뭘 가져올지 미리 정해둔 규칙"이라는 감을 잡았는가
- 터미널을 열고, `mkdir`로 폴더를 만들고, `cd`로 그 폴더까지 이동할 수 있는가

- 터미널, 에디터, 언어가 각각 다른 역할이라는 걸 구분할 수 있는가
-

요약

- 홈페이지는 프론트엔드(화면) - 백엔드(서버) - DB(데이터 저장소), 세 조각이다.
 - 화면이 요청을 보내면 서버가 처리해 응답으로 돌려준다. 우리가 쓰는 모든 웹은 이 왕복의 반복이다.
 - 세 조각 구조를 내 말로 설명해보고, 실제 사이트 기능에 대입해보면 이해했는지 확인할 수 있다.
 - API는 "이 주소로 요청이 오면 DB를 이렇게 다룬다"고 미리 정해둔 규칙이다. 같은 주소라도 저장 요청과 조회 요청은 서로 다른 규칙으로 따로 정해져 있다.
 - 브라우저는 DB에 직접 접속하지 않고 항상 서버를 거친다. 서버가 요청을 걸러주는 관문 역할을 하기 때문이다.
 - 개발환경은 터미널(명령을 내리는 창), 에디터(코드를 보는 곳), 언어(코드를 적는 문법)로 나뉜다. 이 위에 Gemini API 키, Supabase, Vercel, Railway 같은 도구가 앞으로 순서대로 더해진다.
 - 코드를 적는 것과 그 코드를 서버로 여는 것은 별개다. 서버 실행은 3주차-1에서 배운다.
-

문제 풀러 가기

이번 구간 내용을 제대로 이해했다면 풀 수 있는 문제 3개입니다. 먼저 혼자 풀어보고 아래 정답을 확인하세요.

1. 다음 중 클라이언트와 서버 사이에 오가는 요청/응답에 대한 설명으로 옳은 것을 고르시오.
 - ① 브라우저(클라이언트)가 서버에게 정보를 달라고 보내는 것을 요청(request)이라 하고, 서버가 처리해서 돌려주는 것을 응답(response)이라 한다
 - ② 서버가 브라우저에게 먼저 정보를 요청하고, 브라우저가 그 요청에 응답한다
 - ③ 요청은 사용자가 버튼을 누를 때만 생기고, 화면이 열리자마자 자동으로 나가는 요청은 없다
 - ④ DB가 서버에게 요청을 보내고 서버가 그 요청에 응답한다
2. 정민의 AI 한줄일기에서 "지난 일기 목록을 보여줘"라는 요청이 왔을 때, 이 요청을 받은 서버가 하는 일로 옳은 것을 고르시오.
 - ① DB(창고)를 열어 기록을 꺼내온 뒤, 그것을 응답에 담아 클라이언트(화면)로 돌려준다
 - ② 클라이언트가 직접 DB에 접속해서 기록을 꺼내오고, 서버는 아무 일도 하지 않는다
 - ③ 서버는 DB 없이 자체적으로 기록을 기억하고 있다가 돌려준다
 - ④ 서버는 일기를 저장할 때만 DB를 열고, 목록을 보여줄 때는 브라우저에 남아 있는 기록을 그대로 쓴다
3. 다음 중 틀린 것을 고르시오.
 - ① 터미널은 컴퓨터에게 글자로 명령을 내리는 창이다
 - ② 코드 에디터는 코드를 사람이 보기 편하게 해주는 도구일 뿐, 메모장으로 수정해도 코드는 똑같이 돌아간다
 - ③ 언어로 코드를 다 적으면 그 코드는 저절로 서버에서 돌아간다
 - ④ API는 "이 주소로 요청이 오면 DB에서 뭘 가져올지"를 미리 정해둔 규칙이다

정답

1. ① - 클라이언트가 서버에 요청을 보내면, 서버는 처리 후 응답으로 돌려준다.
 2. ① - 서버는 필요하면 DB(창고)를 열어보고, 처리한 결과를 응답에 담아 클라이언트로 돌려준다.
 3. ③ - 언어로 코드를 적는 것과 그 코드를 실제로 돌아가게 여는 것(서버 실행)은 별개다. 코드를 다 적어도 서버로 열어주지 않으면 저절로 돌아가지 않는다.
-